Problem Set Solutions: Seminar 04

Lists, Stacks, and Queues
MIT-style solution format

Algorithms and Data Structures I

June 29, 2026

Course:	Algorithms and Data Structures I
Topic:	Linked lists, stacks, queues, deques, monotone structures
Style:	Full problem-set solutions with algorithms, proofs, and complexity bounds

Why this matters. The point of stacks, queues, and linked lists is not only that they support restricted updates. Their real algorithmic value is that the restriction creates strong order invariants. A monotone stack turns nearest-greater and histogram problems into one linear scan; a monotone deque turns sliding-window minima into amortized constant time; an explicit stack removes recursion while preserving the same divide-and-conquer state; and pointer rewiring gives efficient algorithms on linked lists without array-style random access. The recurring theme is to store only the candidates that can still matter.

Contents

Conventions. Arrays are indexed from 1 unless stated otherwise. Intervals $[l, r]$ are inclusive. For stacks, top denotes the top element. All comparisons in nearest-greater/smaller problems are strict. For large counters and sums, use a sufficiently wide integer type.

Problem 1. Nonrecursive merge sort and quicksort

Show how to implement (a) merge sort and (b) quicksort without recursion.

Solution.

(a) Bottom-up merge sort.

Idea. Recursive merge sort first sorts runs of length 1, then combines them into runs of length 2, then length 4, and so on. We can perform these levels iteratively.

Algorithm. Let w be the current run length. Start with $w = 1$. While $w < n$, scan the array from left to right and merge adjacent sorted runs

$$[l, l + w - 1] \quad \text{and} \quad [l + w, l + 2w - 1]$$

with endpoints truncated at n. After finishing the scan, set $w \leftarrow 2w$. Alternate between the original array and an auxiliary array, or copy merged runs back after each level.

Correctness. Initially each run of length 1 is sorted. Suppose that at the beginning of some level all runs of length w are sorted. The standard merge operation takes two adjacent sorted runs and produces their sorted concatenation, so after the level all runs of length $2w$ are sorted. By induction over the levels, after the first level with $w \geq n$ the whole array is sorted.

Running time. Each level merges a total of n elements. There are $\lceil \log_2 n \rceil$ levels. Therefore the time is $O(n \log n)$ and the auxiliary memory is $O(n)$ for an array implementation.

(b) Iterative quicksort.

Idea. The recursive quicksort call stack stores only intervals still waiting to be sorted. Replace the program stack by an explicit stack of intervals.

Algorithm. Push the initial interval $[1, n]$ onto a stack. While the stack is nonempty, pop an interval $[l, r]$. If $l \geq r$, ignore it. Otherwise choose a pivot, partition $a_l, \ldots, a_r$ into elements smaller than the pivot and elements greater than the pivot, and obtain two subintervals. Push those subintervals onto the stack.

To keep the stack depth small in practice, after partitioning process the smaller subinterval immediately and push only the larger one, or equivalently push the larger one first and the smaller one second. This gives $O(\log n)$ stack size under balanced partitions; the time bounds are the usual quicksort bounds.

Correctness. The partition step places every element of the current interval into the correct side relative to the pivot, and the pivot position is final. The only remaining disorder is inside the two subintervals. Thus replacing a recursive call on an interval by pushing that interval onto an explicit stack preserves exactly the same set of pending subproblems. When the stack becomes empty, every subproblem has either size at most 1 or has been partitioned until its elements are in sorted order. Hence the whole array is sorted.

Running time. Bottom-up merge sort runs in $O(n \log n)$ time. Iterative quicksort has expected $O(n \log n)$ time with a randomized pivot and worst-case $O(n^2)$ time, exactly as recursive quicksort; it removes recursion by using an explicit stack of intervals.

Problem 2. Nearest greater and smaller elements in linear time

Given an array $a_1, a_2, \ldots, a_n$, for all i find in total $O(n)$ time:

(a) the minimum $j > i$ such that $a_j > a_i$;

(b) the minimum $j > i$ such that $a_j < a_i$;

(c) the maximum $j < i$ such that $a_j > a_i$;

(d) the maximum $j < i$ such that $a_j < a_i$.

Solution.

Idea. A nearest-greater or nearest-smaller query is local in index but global in value. A monotone stack keeps exactly the indices that have not yet been blocked by a closer and more useful candidate.

(a) Next greater element. Scan $i = n, n-1, \ldots, 1$. Maintain a stack of indices whose values are strictly increasing from top to bottom; equivalently, the top is the nearest currently useful candidate. Before answering i, pop while $a_{\text{top}} \le a_i$. Then the top, if it exists, is the minimum $j > i$ with $a_j > a_i$. Push i.

(b) Next smaller element. Again scan from right to left. Pop while $a_{\text{top}} \ge a_i$. The remaining top, if it exists, is the minimum $j > i$ with $a_j < a_i$. Push i.

(c) Previous greater element. Scan $i = 1, 2, \ldots, n$. Pop while $a_{\text{top}} \le a_i$. The remaining top, if it exists, is the maximum $j < i$ with $a_j > a_i$. Push i.

(d) Previous smaller element. Scan from left to right. Pop while $a_{\text{top}} \ge a_i$. The remaining top, if it exists, is the maximum $j < i$ with $a_j < a_i$. Push i.

Correctness. We prove the case of the next greater element; the other three are identical with the direction and inequality changed. While scanning from right to left, suppose an index t on the stack has $a_t \le a_i$. Then t cannot be the answer for i, and it cannot be the answer for any earlier index $h < i$ for which i is closer than t and $a_i \ge a_t$. Thus t is dominated and can be safely removed. After all dominated indices are removed, the stack top is greater than a_i. All indices above it were impossible, and all closer greater candidates would have remained above it. Therefore the top is exactly the nearest greater index to the right.

Running time. Each index is pushed once and popped at most once in each scan. Each of the four computations takes $O(n)$ time and $O(n)$ memory; all four together are still $O(n)$ time and $O(n)$ memory.

Problem 3. A queue with minimum via a deque

Propose a method for storing the minimum value in a queue using a double-ended queue. If n operations are performed, the total running time must be $O(n)$.

Solution.

Idea. The front of the minimum-deque stores the current minimum. Values larger than a newly inserted value can never again become minimum before the new value leaves the queue, so they should be deleted immediately.

Algorithm. Store queue elements with increasing timestamps. Maintain an ordinary FIFO queue Q and an auxiliary deque D of pairs $(value, time)$ whose values are nondecreasing from front to back.

- **push**(x): insert (x, t) into Q. While D is nonempty and its last value is greater than x, delete the last element of D. Then append (x, t) to D.
- **pop**: remove the front pair (x, t) from Q. If the front of D has timestamp t, remove it from D as well.
- **minimum**: return the value at the front of D.

Correctness. The deque D contains queue elements that are not dominated by a later element of smaller or equal value. If a value $y > x$ stands behind the new value x in time order, then y will leave the queue no earlier than x and is larger than x, so y can never be the minimum while x is present. Hence removing such values from the back is safe. The remaining deque is ordered by queue time and has nondecreasing values, so its front is the smallest value among all nondominated queue elements. Every deleted dominated element is larger than some later element still in the queue, and therefore cannot be the queue minimum. Thus the front of D is exactly the minimum of Q.

Running time. Every inserted element is appended to D once and removed from D at most once. Therefore n queue operations take total $O(n)$ time, i.e. amortized $O(1)$ time per operation. The memory is $O(n)$ in the worst case.

Problem 4. A stack with add-to-all in $O(1)$

Design a stack that can add an arbitrary correction x to every stored value in $O(1)$ time.

Solution.

Idea. Separate the common offset from the individual stored values. Store values in coordinates relative to a global shift.

Algorithm. Maintain a number Δ, initially 0. Instead of storing an actual value v, store $v - \Delta$.

- **push**(v): push $v - \Delta$.
- **top**: return top $+\Delta$.
- **pop**: pop the stored value y and return $y + \Delta$.
- **addAll**(x): set $\Delta \leftarrow \Delta + x$.

Correctness. At all times, the actual value represented by a stored number y is $y + \Delta$. A push of v stores $v - \Delta$, whose represented value is v. Increasing all actual values by x is exactly the same as increasing the common offset Δ by x, because every represented value changes from $y + \Delta$ to $y + \Delta + x$. The top and pop operations reconstruct the actual value by adding the current offset. Hence all operations behave exactly as required.

Running time. Each operation performs only constant many arithmetic and stack operations. Therefore all operations run in $O(1)$ time. The memory is linear in the number of stored stack elements.

Problem 5. Sorting a linked list in $O(n \log n)$ time and $O(1)$ extra memory

Show how to sort a linked list of length n in $O(n \log n)$ time using $O(1)$ additional memory.

Solution.

Idea. Array quicksort needs random access; linked lists do not provide it. Merge sort is natural for linked lists because merging sorted lists is just pointer rewiring. To avoid recursion, use a bottom-up merge sort.

Algorithm. Use iterative run lengths $w = 1, 2, 4, \ldots$. At the beginning of a phase, the list is partitioned into sorted runs of length w except possibly the last run. Scan the list and repeatedly detach two consecutive runs of length at most w, merge them by changing next-pointers, and attach the merged run to the answer list for this phase. Then double w.

The merge of two sorted linked lists uses only a constant number of pointers: compare their heads, append the smaller head to the output, and advance that list. No new list nodes are allocated.

Correctness. Initially every single node is a sorted run. Suppose that before a phase all runs of length w are sorted. The merge procedure outputs the sorted union of two adjacent sorted runs while preserving all their elements. Therefore after the phase, every run of length $2w$ is sorted. By induction over the phases, once $w \geq n$, the whole list is one sorted run. Since the algorithm only changes next-pointers and never changes the multiset of nodes, the final list is exactly the sorted version of the input list.

Running time. Each phase scans and rewires all n nodes once, so it costs $O(n)$. There are $O(\log n)$ phases. The total time is $O(n \log n)$. The algorithm uses $O(1)$ auxiliary memory besides the input nodes.

Problem 6. Maximum subarray minimum times length

Given numbers $a_1, \ldots, a_n$, find

$$\max_{1 \le l \le r \le n} \left(\min_{i \in [l,r]} a_i \right) (r - l + 1).$$

Solution.

Idea. This is the largest-rectangle-in-a-histogram problem. If a_i is the minimum of the chosen segment, then the best segment using a_i as the limiting height extends left and right until the first strictly smaller element.

Algorithm. For every i, compute

$$L_i = \max\{j < i : a_j < a_i\}, \qquad R_i = \min\{j > i : a_j < a_i\},$$

where absent values are treated as $L_i = 0$ and $R_i = n+1$. These arrays are computed by monotone stacks:

- scan left to right, popping while $a_{\text{top}} \ge a_i$, to get L_i;
- scan right to left, popping while $a_{\text{top}} \ge a_i$, to get R_i.

Then return

$$\max_{1 \le i \le n} a_i \, (R_i - L_i - 1).$$

Correctness. For a fixed i, every element in the interval (L_i, R_i) has value at least a_i, and the endpoints L_i and R_i, if they exist, have value strictly smaller than a_i. Therefore the longest segment on which a_i can be the minimum is exactly $[L_i + 1, R_i - 1]$, and its contribution is $a_i(R_i - L_i - 1)$.

Now take an optimal segment $[l, r]$, and let i be an index where the minimum on this segment is attained. Then all elements of $[l, r]$ are at least a_i, so no strictly smaller element lies inside $[l, r]$. Hence $L_i < l$ and $R_i > r$, so

$$R_i - L_i - 1 \ge r - l + 1.$$

The candidate computed for i is therefore at least the value of the optimal segment. Since every computed candidate is itself realized by a valid segment, the maximum of the candidates is exactly the answer.

Running time. Each index is pushed and popped at most once in each stack scan. The total time is $O(n)$ and the memory is $O(n)$.

Problem 7. Correct bracket substrings of fixed length

Given a string of opening and closing parentheses of length n and a number k, determine the number of substrings of length k that are correct bracket sequences. The required time is $O(n)$.

Solution.

Idea. A bracket sequence is correct exactly when its total balance is zero and no prefix has negative relative balance. This can be checked for every sliding window using prefix sums and a sliding minimum.

Algorithm. If k is odd, return 0. Define prefix balances

$$p_0 = 0, \qquad p_i = p_{i-1} + \begin{cases} 1, & s_i = (, \\ -1, & s_i =). \end{cases}$$

For a substring $s_l \dots s_r$, where $r = l + k - 1$, it is correct iff

$$p_r = p_{l-1} \quad \text{and} \quad \min_{t \in [l,r]} p_t \geq p_{l-1}.$$

Maintain the minimum of p_t over the sliding index window $t \in [l, l+k-1]$ with a monotone deque. For each $l = 1, \dots, n-k+1$, test the two conditions above and count successes.

Correctness. The balance of the whole substring $s_l \dots s_r$ is $p_r - p_{l-1}$, so total balance zero is equivalent to $p_r = p_{l-1}$. A prefix $s_l \dots s_t$ of the substring has relative balance $p_t - p_{l-1}$. Therefore no prefix has negative balance exactly when $p_t \geq p_{l-1}$ for every $t \in [l, r]$, i.e. when the minimum prefix balance in that window is at least p_{l-1}. The algorithm checks precisely these two necessary and sufficient conditions for every length-k substring.

Running time. The prefix array is computed in $O(n)$ time. The sliding minimum deque pushes and pops each index at most once, so all window minima take $O(n)$ time. The memory is $O(n)$, or $O(k)$ if prefix values are streamed carefully.

Problem 8. Can one push order realize a required pop order?

Two arrays $a_1, \dots, a_n$ and $b_1, \dots, b_n$ contain pairwise distinct numbers. Starting with an empty stack, pushes must occur in the fixed order

$$\text{push}(a_1), \text{push}(a_2), \dots, \text{push}(a_n).$$

Can we insert pop operations so that the popped elements are exactly $b_1, \dots, b_n$ in that order?

Solution.

Algorithm. Simulate the only greedy possibility. Let $j = 1$ be the next required element of b. For $i = 1, 2, \dots, n$, push a_i. After each push, while the stack is nonempty and its top is b_j, pop it and increment j. At the end, answer yes iff $j = n + 1$.

Correctness. Whenever the stack top equals the next required output b_j, popping it immediately is forced in the following sense: delaying this pop cannot make any other element below it accessible, and future pushes will only be placed above it. Therefore any valid sequence can be modified to perform this pop immediately without changing the resulting pop order. Conversely, if the top is not b_j, then popping is impossible in any valid sequence, because it would output the wrong element. Thus the greedy simulation performs exactly all forced pops and never performs an invalid pop. If it outputs all of b, a valid insertion of pops has been constructed. If it gets stuck before outputting all of b, no valid sequence exists.

Running time. Each element is pushed once and popped at most once. The simulation takes $O(n)$ time and $O(n)$ memory.

Problem 9. Evaluating reverse Polish notation and ordinary notation

Given a correct expression in reverse Polish notation, evaluate it in time linear in its length, assuming each arithmetic operation takes $O(1)$ time. What should be done if the expression is written in the usual infix form?

Solution.

Reverse Polish notation.

Algorithm. Scan tokens from left to right. Maintain a stack of numbers. If the token is a number, push it. If the token is a binary operation $\circ$, pop the right operand y, then the left operand x, compute $x \circ y$, and push the result. At the end, the stack contains exactly one number; this is the value of the expression.

Correctness. In reverse Polish notation, every operator appears after all operands of its subexpression. Thus when an operator is scanned, the values of its two operand subexpressions have already been computed and are on top of the stack in the correct order. Replacing these two values by the result of the operation preserves the invariant that the stack contains values of already completed subexpressions whose parent operators have not yet been processed. At the end the whole expression is completed, so the single remaining value is the answer.

Traditional infix notation. Use Dijkstra's two-stack algorithm, or first convert the infix expression to reverse Polish notation by the shunting-yard algorithm. Maintain an operator stack and a value/output stack. Parentheses and operator precedence determine when operators are popped from the operator stack. Since each token is pushed and popped only a constant number of times, the evaluation or conversion remains linear.

Running time. For both RPN evaluation and infix evaluation with stacks, the running time is $O(m)$ for m tokens and the memory is $O(m)$ in the worst case.

Problem 10. Checking an XML string

Check whether a given string is a correct XML string, for example

`<a><b></b><uv></uv></a>`.

Solution.

Idea. XML correctness is a bracket-matching problem where each opening bracket carries a name. A closing tag must match the most recent unmatched opening tag.

Algorithm. Scan the string from left to right and parse tags of the form `<name>` and `</name>`. Maintain a stack of open tag names and a counter of root elements.

- On an opening tag `<name>`, require that `name` is nonempty and contains only allowed name characters. If the stack is empty, increment the root counter and require it not to exceed 1. Push `name`.
- On a closing tag `</name>`, require the stack to be nonempty and its top to equal `name`; then pop.
- Any malformed tag, missing bracket, empty name, or text outside the allowed grammar makes the string invalid.

At the end, the string is correct iff the stack is empty and exactly one root tag was opened.

Correctness. The stack stores exactly the chain of currently open tags from the root to the current position. When an opening tag is read, it becomes the most recent unmatched tag, so pushing it preserves the invariant. When a closing tag is read, XML nesting requires it to close the most recent unmatched opening tag; this is exactly the stack top. If the names differ or the stack is empty, no valid XML nesting is possible. If the scan ends with a nonempty stack, some tags were never closed; if more than one root was opened, the string is not a single XML document. Therefore the algorithm accepts exactly the correct XML strings.

Running time. Each character is read and parsed a constant number of times. The running time is $O(n)$ and the stack memory is $O(n)$ in the worst case.

Problem 11. Recovering the original array after inserting equal adjacent pairs

An initial array had all adjacent elements different. Several times the following operation was applied: insert two equal numbers next to each other. Given the final array, recover the initial array.

Solution.

Idea. The reverse operation is to delete two adjacent equal elements. The normal form under the rule $xx \to \varepsilon$ is computed by a stack.

Algorithm. Scan the final array from left to right. Maintain a stack.

- If the stack is nonempty and its top equals the current value x, pop the top.
- Otherwise push x.

After the scan, output the stack from bottom to top.

Correctness. The stack algorithm performs exactly the standard reduction rule "delete two adjacent equal elements" as soon as such a pair is created at the end of the processed prefix. We need to show that this recovers the unique original array.

The rewriting rule $xx \to \varepsilon$ is terminating because every deletion shortens the sequence by 2. It is also locally confluent: the only possible overlap of two deletable pairs has the form xxx, and deleting the left pair or the right pair leaves the same one-element sequence x; disjoint deletions clearly commute. Hence the final irreducible form is independent of the order of deletions.

The initial array had no equal adjacent elements, so it is irreducible. The final array was obtained from it by inserting adjacent equal pairs, so reversing those insertions reduces the final array to the initial one. By uniqueness of the irreducible form, every complete deletion process gives the same result. The stack algorithm computes this irreducible form, and therefore outputs the original array.

Running time. Each input element is pushed at most once and popped at most once. The running time is $O(n)$ and the memory is $O(n)$.

Problem 12. Detecting a cycle in a singly linked list

Given a singly linked list with a pointer to the head, possibly cyclic, determine whether it has a cycle in $O(n)$ time and $O(1)$ extra memory.

Solution.

Algorithm. Use Floyd's tortoise-and-hare algorithm. Maintain two pointers:

- slow moves by one edge per step;
- fast moves by two edges per step.

If fast reaches a null pointer, the list is acyclic. If slow and fast ever become equal, the list contains a cycle.

Correctness. If the list is acyclic, following next-pointers eventually reaches null. The fast pointer moves along the same path faster than the slow pointer, so it reaches null after $O(n)$ steps; the algorithm correctly reports no cycle.

If the list has a cycle, both pointers eventually enter the cycle. Once both are inside, consider their positions modulo the cycle length c. At each iteration, fast gains one position on slow modulo c. Therefore the modular distance between them decreases by 1 modulo c each step and must become 0 within at most c further iterations. They meet, and the algorithm correctly reports a cycle.

Running time. Before entering the cycle, each pointer traverses at most the noncyclic prefix. After that, they meet within one cycle length. Thus the time is $O(n)$ and the extra memory is $O(1)$.

Problem 13. Yacht production, sales, and storage cost

A company can produce at most d yachts per month. It works for n months. At the end of month i, a_i customers arrive; each wants to buy one yacht. If no yacht is available, the customer leaves forever. Storage is unlimited, but storing one yacht for one month costs 1. Determine the maximum number of yachts that can be sold and the minimum storage cost needed to achieve that maximum. Required time: $O(n)$.

Solution.

Idea. There are two logically separate questions. First, maximize sales: selling a yacht earlier never decreases the maximum possible number of sales, because a customer who leaves cannot be recovered. Second, after the optimal sales quantities per month are fixed, minimize storage by producing the sold yachts as late as their sale deadlines allow.

Algorithm. First compute the maximum possible sales and choose the earliest possible optimal sales sequence. Let *stock* be the number of available yachts in the hypothetical schedule that produces d yachts every month.

$$stock \leftarrow 0, \qquad S \leftarrow 0.$$

For $i = 1, 2, \ldots, n$:

1. $stock \leftarrow stock + d$;
2. $s_i \leftarrow \min(a_i, stock)$;
3. $stock \leftarrow stock - s_i$;
4. $S \leftarrow S + s_i$.

Here s_i is the number of customers served in month i, and S is the maximum number of sales.

Now compute the minimum storage cost for the fixed demand sequence $s_1, \ldots, s_n$. Let

$$A = \sum_{i=1}^{n} i\, s_i$$

be the sum of sale months over all sold yachts. We maximize the sum of production months by producing as late as possible. Scan months backward:

$$need \leftarrow 0, \qquad B \leftarrow 0.$$

For $i = n, n-1, \ldots, 1$:

1. $need \leftarrow need + s_i$;
2. $q_i \leftarrow \min(d, need)$;
3. $B \leftarrow B + i\, q_i$;
4. $need \leftarrow need - q_i$.

Then the minimum storage cost is

$$A - B.$$

Return $(S, A - B)$.

Correctness. First we prove maximality of S. After month i, no algorithm can have sold more than id yachts, because only i months of production capacity have occurred. Also, in month i itself, at most a_i customers can be served. The forward greedy algorithm always sells as many yachts as are available to current customers. If it leaves some customer unserved in month i, then all yachts that could have been produced by that time have already been sold, so no other schedule can have larger cumulative sales through month i. By induction over months, the greedy cumulative number of sales after every prefix is maximum possible. In particular, S is the maximum possible total number of sales.

Among all maximum-sale schedules, the forward pass chooses sales as early as possible: after every prefix, it has the largest possible number of sales. Moving one sale from a later month to an earlier month where an unserved customer exists cannot increase storage cost: the sale month decreases, and the production month needed for that yacht can decrease by no more than the same amount, since production cannot occur after the sale month. Therefore an earliest maximum-sale sequence is storage-optimal among maximum-sale sequences once production is chosen optimally.

It remains to prove the backward production rule. For the fixed sales quantities s_i, every sold yacht has a deadline equal to its sale month. Scanning backward, *need* is the number of sold yachts with deadlines at least i that have not yet been assigned to a production month. Producing $q_i = \min(d, need)$ yachts in month i uses as much as possible of the latest still available production capacity. This is optimal: any yacht not produced in month i must be produced in an earlier month, which can only weakly increase storage cost. By induction from n down to 1, the backward pass maximizes the sum B of production months among all feasible schedules for the fixed sales sequence.

For each sold yacht, storage time equals

$$\text{sale month} - \text{production month}.$$

Summing over all sold yachts gives total storage cost $A - B$. Since the algorithm minimizes sale months among maximum-sale schedules and maximizes production months for those sales, it achieves the minimum possible storage cost subject to selling the maximum possible number of yachts.

Running time. Both passes are linear. The running time is $O(n)$, and the memory is $O(n)$ if the array s_i is stored. If the forward sales sequence is written to disk or recomputed from saved prefix data, memory can be reduced, but $O(n)$ memory already satisfies the requirement.